

Google for Startups

AI Agents Challenge

The complete resource guide

2026

The complete resource guide

Welcome to the Google for Startups AI Agents Challenge! This challenge is designed specifically to give startup teams the dedicated support, credits, and technical resources needed to build production-ready agents using Gemini Enterprise Agent Platform (Agent Platform).

All eligible startups who create a project will receive \$500 in credits and a chance to win a share of a \$90,000 prize pool.

Sign up for Google Cloud

Existing users can visit cloud.google.com/ and create a new project. If you are new to Google Cloud, create your account through these steps:

1. **Sign in:** Go to cloud.google.com, click Get started for free, and log in with your Gmail account.
2. **Initial setup:** Select your country, agree to the Terms of Service, and click Continue.
3. **Verify identity:** Select Individual as your account type. Enter your name, address, and payment method (required to block bots; you will not be auto-charged), then click Start My Free Trial.
4. **Initiate project:** In the Cloud Console, click the project drop-down menu in the top-blue navigation bar and select New Project.
5. **Create project:** Enter a project name (e.g., Startup-AI-Challenge-app), leave Organization set to No organization, and click Create.

Three specialized tracks for every stage of development

To meet startups where they are—whether you are just starting a project or ready to scale—we have structured the competition around three core tracks.

Track 1

Build (Net-new agents)

For those starting with a blank canvas and a complex business problem. Participants will leverage the Agent Development Kit (ADK)—or preferred frameworks like LangChain or CrewAI—to architect autonomous agents that use the Model Context Protocol (MCP) to securely connect to external tools.

Accessing and bootstrapping agent tools on GCP

To move fast and avoid building from scratch, use Google's Agent Development Kit (ADK) and community templates to instantly bootstrap your project.

- **Understand the Core ADK Framework:** Review the base [code-first toolkit](#) to understand how Agents, Sessions, and Runners work together.
- **Bootstrap via the Agents CLI:** The fastest path to a working prototype. Use the newly launched [Agents CLI](#) to instantly scaffold and clone production-ready templates for ReAct, RAG, and Multi-Agent patterns right from your terminal.
- **Access the Agent Starter Pack repository:** If you prefer a manual approach over the CLI, you use the official [Agent Starter Pack](#) repository to access foundational code for ReAct, RAG, and Multi-Agent patterns.
- **Explore pre-configured examples:** Need inspiration? Browse this curated community list of 80+ hackathon-winning and pre-configured agents (e.g., customer support, data analytics, structured output) [Awesome ADK Agents](#).
- **Quick setup in Cloud Shell:** You don't need a local environment. Open Cloud Shell in your GCP console, run `git clone` on your chosen template, configure your API keys, and test your agent directly in the browser before deploying.

Learn how to build agents on google-adk

To see exactly how developers are building, orchestrating, and deploying agents using Google's Agent Development Kit, explore the following code repositories and documentation:

- **Official Python ADK repository ([google/adk-python](#)):** The primary source of truth for the Python framework. The main branch contains essential snippets for setting up the 5-layer architecture, defining agent logic, utilizing pre-built tools (like the MCP and OpenAPI specs), and running the `AgentEngineSandboxCodeExecutor`.
- **Official Agent Platform documentation:** Google Cloud's official quickstarts and code snippets. This documentation bridges the gap between local code and cloud deployment, showing how to connect ADK configurations to a runtime for effortless scaling.
- **ADK Agents community collection:** A massive, curated [GitHub repository](#) featuring over 80 production-ready agent templates and hackathon-winning projects. This is the best place for participants to find highly specific, functional snippets for things like "Structured Output Agents" or "Multi-Agent Orchestration."
- **Agent Search and grounding notebooks:** For participants building in the RAG or Optimization tracks, this official [Google Cloud repository](#) provides Jupyter Notebooks with snippets on how to securely ground ADK agent responses using Datastores and Google Search.

Deploying and integrating agents with Agent Platform tools

Once you have bootstrapped your agent code, the next step is deploying it to Google Cloud's managed infrastructure and connecting it to your enterprise data. Here is the official documentation you will need to implement your ADK code into Google's agentic ecosystem:

- **Deploy to Agent Runtime:** Learn how to take your local ADK code and deploy it to a fully-managed runtime. This [documentation](#) covers scaling, IAM security configuration, and customizing your agent's container image to transition from prototype to production.
- **Grounding and custom RAG (Agent Search):** Connect your agent to Agent Search Data Stores to prevent hallucinations. This allows you to ground your agent's responses using enterprise data stored natively in Cloud Storage or BigQuery.
- **Managing state (Sessions and Memory Bank):** A production agent requires contextual memory. This [guide](#) details how to configure your ADK AdkApp class to use Agent Runtime Sessions and Memory Banks to store individual interactions and retain long-term user preferences.
- **Public grounding (Google Search):** For agents requiring real-time public data, this [documentation](#) shows how to utilize the "Grounding with Google Search" tool. It provides API request configurations and explains how to display compliant search suggestions directly to your users.

Example

The concept

A Smart Facility Energy Agent, a multi-agent system designed to help commercial property managers dynamically optimize their building's energy consumption based on occupancy, weather forecasts, and grid pricing.

The build

Move from a blank canvas to a functional prototype that connects to external tools and executes tasks autonomously. The startup uses the Agent Development Kit (ADK) to build the core orchestration engine. The agent utilizes the Model Context Protocol (MCP) to securely connect to the building's existing IoT HVAC sensors and real-time local weather APIs.

The action

Instead of just reporting data, the agent moves from static code to declarative intent. It autonomously lowers heating in unoccupied conference rooms based on scheduled calendar data.

Optimize (Existing agents)

Designed for teams with existing agents that need to transition from sandbox experimentation to real-world reliability. This track focuses on stress-testing multi-step reasoning, debugging stalled logic, and refining system instructions to achieve production-grade scale.

Testing and refining your agent

To transition your agent from a functional prototype to a reliable solution, you must rigorously test its logic against unpredictable real-world scenarios. We encourage participants to guarantee quality with Agent Simulation, Agent Evaluation, and Agent Observability. These tools provide full execution traces and a real-time lens into agent reasoning to help ensure your agents always hit their goals.

- **Framework flexibility (Mix and match):** Agent Platform services (Simulation, Observability, Runtime, etc.) are designed to work with any agent framework. While they integrate most seamlessly with the Agent Development Kit (ADK), developers have total freedom. You can use the ADK standalone without these services, or mix and match these enterprise services with your preferred framework (like LangChain or CrewAI).
- **The Agents CLI:** The easiest way for a developer to build, evaluate, and iterate on these optimizations is using the newly launched [Agents CLI](#). This is the next-generation tool for agent workflows (replacing the previous agents-starter-pack).
- **Agent Simulation and Evaluation:** Don't wait for real-world failures. Use the [Evaluation and Simulation documentation](#) to learn how to generate synthetic user interactions and multivariable edge cases. Measure your agent's performance against established baselines to stress-test your agent in a controlled environment.
- **Agent Observability:** Implement observability to visually trace complex reasoning and debug decision-making bottlenecks.
- **Managing State (Sessions, Runtime and Memory Bank):** A production agent requires contextual memory to handle complex corrections. Configure Agent Runtime Sessions to ensure your agent maintains context during long, multi-turn evaluations. Check out the [ADK Quickstart guide](#) to see how easily these stateful services integrate.

Deploying and submitting your optimized agent

Once your agent has been rigorously tested and refined using simulation and observability tools, the final step is deploying it to a stable infrastructure so you can record your demo and submit your project. Because Track 2 focuses on real-world reliability, your deployment should emphasize security, scale, and enterprise data integration.

- **Deploy to Agent Runtime:** Take your optimized code and deploy it to Google Cloud's fully-managed Agent Runtime. Review the [documentation](#) on scaling and IAM security configuration to ensure your deployed agent is robust enough to handle the complex scenarios you simulated.
- **Grounding (Custom RAG):** Securely connect your agent to Agent Search Data Stores. Ground your agent's responses using real (or mock) data stored natively in Cloud Storage or BigQuery to demonstrate production-grade reliability.
- **Public grounding (Google Search):** If your edge-case testing revealed the need for real-time data (like the sudden weather event in our Energy Agent example), implement the "Grounding with Google Search" tool to allow your agent to pull compliant, up-to-the-minute public information.
- **Capture your demo:** Build a simple frontend (or use the Cloud console UI) to showcase your agent in action. Tip for Track 2 submissions: Your demo video should clearly highlight the "before and after." Show the judges exactly how your optimized agent now successfully navigates the complex, multivariable edge cases that would have previously caused it to stall or fail!

Example

The concept

Take an agent that works perfectly in a sandbox and apply rigorous engineering discipline to make it handle real-world edge cases.

The build

The team uses Agent Simulation to generate synthetic user interactions and test the agent against these rare, multivariable events in a controlled environment. By leveraging Agent Observability, the startup can visually trace the agent's complex reasoning to debug stalled logic where the agent couldn't decide whether to prioritize occupant comfort or energy cost. They then use Agent Optimizer to programmatically refine the system instructions to handle the conflict seamlessly in the future.

The action

The Smart Facility Energy Agent works well on a normal day, but struggles when a sudden extreme weather event occurs at the exact same time a utility provider issues a peak-demand pricing surge.

Track 3

Refactor for Google Cloud Marketplace and Gemini Enterprise

Transform your functional MVP into a scalable, monetizable asset ready for the enterprise market. This track involves migrating runtimes to Google Cloud and refactoring architectures to utilize the **Agent-to-Agent (A2A) protocol** for discovery and coordination within the Google Cloud Agent Marketplace.

****Important Note on Marketplace Listing:** Completing Track 3 ensures your agent meets the foundational technical criteria for Gemini Enterprise. However, submission to this challenge does not guarantee a listing on the Google Cloud Marketplace. Final publication requires further business and security evaluations, as well as executing formal partnership agreements with Google Cloud.*

Step-by-step:

- **Validate your B2B use case:** Your agent must be explicitly designed for business workflows. Adjust any consumer-focused messaging, security posture, and application logic to serve an enterprise buyer.
- **Migrate to a Google Cloud Runtime.** Host your application logic and compute on Google Cloud. Use [Cloud Run](#) for rapidly scaling, stateless, containerized microservices, or [GKE](#) for complex orchestration and stateful workloads.
- **Route LLMs through [Model Garden](#).** Ensure strict data security by powering your reasoning engine with Gemini or a third-party/open-source LLM deployed specifically through Model Garden.
- **Implement the [A2A Protocol](#).** Refactor your architecture to be A2A-native. This allows your agent to securely authenticate and exchange data with other agents in a customer's environment.
- **Multi-agent orchestration ([Agent Builder overview](#)):** Use this [overview](#) to understand how to securely connect your ADK agent to other enterprise ecosystems using the Agent2Agent protocol and Agent Garden tools.
- **Finalize documentation.** Provide comprehensive documentation mapping your GKE/Cloud Run deployment architecture, detailing your model usage and data processing, and outlining the specific A2A intents your agent exposes and consumes.

Build example #1

The startup refactors the agent to run natively on Google Cloud infrastructure, deploying it via Cloud Run or Google Kubernetes Engine (GKE) to ensure effortless scale. They ensure the underlying reasoning engine is exclusively powered by Gemini. To guarantee the agent is secure by design, they implement Agent Identity to assign a unique cryptographic ID to the agent. The team implements the Agent-to-Agent (A2A) protocol. This allows their Energy Agent to seamlessly communicate with a company's internal HR Agent to anticipate building occupancy spikes during all-hands meetings and pre-cool the building during off-peak hours.

Build example #2

The agent (example: marketing agent) runs natively on Google Cloud infrastructure, utilizing Cloud Run or GKE to handle the high-compute demands of multi-modal video assembly and ensuring effortless scale. The underlying reasoning and generation engine is powered by Gemini, allowing the agent to analyze PDF briefs, generate storyboards, and orchestrate audio-visual assets in a single workflow. The Agent utilizes the Agent-to-Agent (A2A) protocol, allowing the agent to seamlessly communicate with a company's internal Digital Asset Manager (DAM) Agent to retrieve approved brand logos and product imagery, ensuring every generated video remains perfectly on-brand and compliant.

Why participate?



Cloud credits

All eligible startups receive \$500 in credits to fuel their development



Prize pool

Compete for a portion of over \$90,000 in total prizes.



Technical support

Access to Google experts, technical webinars, and dedicated resource guides.



Global recognition

Showcase your work to a panel of expert judges and the broader Google Cloud ecosystem.